CeTTa:
A Direct C Runtime for MeTTa with Verified Translation

Zar Goertzel
Oruži

April 25, 2026

Abstract

MeTTa is a reflective, non-deterministic meta-language with three actively maintained runtimes: Hyperon Experimental (HE, Rust), PeTTa (Prolog), and CeTTa (C). This paper describes CeTTa, a direct C implementation of the HE MeTTa evaluator that passes 297 oracle-verified tests and achieves **9.5× speedup over PeTTa** on genuine depth-10 PLN backward chaining with truth-value propagation (19,845 hypotheses over 1.4M atoms in 15 s vs. PeTTa's 143 s), while losing 13× on pure branching search (8-queens) and 30× on BFS state exploration (tilepuzzle). The runtime features SymbolId-based dispatch (38% instruction reduction), structured space kinds (queue, hash, stack), a pluggable match backend with PathMap/MORK integration, and compiled space artifacts via MORK's `.act` format for zero-copy indexed loading. It is backed by a sorry-free Lean proof (5,400+ lines) establishing semantic correspondence between HE and PeTTa evaluation on the pure fragment, including import commutativity and operational bridges for state and space allocation. A kernel-checked HE′ telescope formalization extends the type system with dependent binder semantics. We present a complete benchmark ladder from shallow evaluation through depth-10 PLN inference, callgrind/massif-based performance analysis, a specification gap analysis, and a three-machine architecture (local search, indexed shared-space, type elaboration) informed by GPT-5.4 Pro architectural review.

1 Introduction

MeTTa [1] is a reflective, typed, non-deterministic language designed as the core language of the OpenCog Hyperon AGI framework. Three implementations exist:

HE Hyperon Experimental (Rust, v0.2.10). The reference implementation with Python bindings and a large standard library.

PeTTa Prolog-based MeTTa (SWI-Prolog). Leverages Prolog's SLD resolution for backward chaining and pattern matching.

CeTTa Direct C evaluator (this work). Targets the HE spec with arena allocation, discrimination-trie indexing, and tail-call optimization.

Contributions.

1. A direct C runtime passing 297 HE-oracle-verified tests.
2. A bidirectional HE↔PeTTa translator (Prolog + executable Lean), with a sorry-free Lean proof of semantic correspondence.

3. Cross-runtime benchmarks showing CeTTa 10–67× faster than HE on shallow evaluation, PeTTa 3× faster on deep proof search, and a callgrind analysis identifying 46% of CeTTa's runtime as optimizable string operations.

4. A characterization of the semantic boundary between HE and PeTTa: the shared fragment (85% of programs) runs identically; the divergence is in free-variable scoping across non-deterministic branches.

5. A specification gap analysis of non-determinism plus mutable state, with 8 concrete examples showing that sequential-per-iteration semantics (CeTTa's default) is correct in 6/8 cases, while HE's interleaved side-effect evaluation loses atoms in real code (the pattern miner).

6. `with-space-snapshot`: a CeTTa extension for branch-isolated mutation, enabling semi-naive forward chaining and speculative search without the side-effect hazards of shared mutable state.

2 CeTTa Architecture

CeTTa is a direct evaluator, not a compiler or boot pipeline. It implements the six mutual functions of the HE evaluation spec (`EvalAtom`, `MettaCall`, `InterpretExpression`, `InterpretFunction`, `InterpretArgs`, `InterpretTuple`) as C functions with a shared arena allocator.

2.1 Core Design

Arena allocation. All atoms are allocated from 64 KB arena blocks. No per-atom `malloc`/`free`. Batch deallocation at scope exit.

Symbol interning. A hash table maps symbol strings to unique pointers, enabling O(1) pointer comparison for symbol equality.

Equation index. Equations `(= lhs rhs)` are indexed by head-symbol hash into 256 buckets, with a discrimination trie within each bucket for fast LHS matching.

Substitution tree. An epoch-based discrimination tree (`SubstTree`) eliminates per-candidate variable renaming during equation queries by tagging indexed variables with monotonic epochs.

Tail-call optimization. A `TAIL_REENTER` trampoline with `goto tail_call` eliminates stack growth for recursive equation dispatch.

Unlimited fuel. Default fuel is −1 (unlimited), matching the HE spec. Stack overflow is caught via `sigaltstack` + `SIGSEGV` handler with a clean error message.

Two-stage stdlib. The standard library is precompiled into a binary blob at build time (`cetta-stage0` → `-compile-stdlib` → blob → `cetta`), giving 6 ms startup.

2.2 Test Suite

CeTTa is validated against the HE CLI (v0.2.10) as oracle. The command `make test` runs each `.metta` test file on both CeTTa and HE, comparing output. Current status: **244 passed, 0 failed, 3 skipped** on the oracle suite, plus 36 profile tests covering module imports, foreign (Python) function dispatch, and extension surfaces.

3 Verified HE↔PeTTa Translation

3.1 Bidirectional Translator

A bidirectional translator converts between HE and PeTTa dialects. It is implemented both as relational Prolog (`he_petta_relational.pl`, with explicit freshness threading) and as computable Lean functions (`translateHE`, `translatePeTTa`).

The translator handles 8 HE→PeTTa rewrite rules (`chain`→`let`, `collapse-bind`→`collapse`, `nop`→`let $fresh X ()`, etc.) and 5 PeTTa→HE rules (`progn`→nested `let`, `prog1`→`let` with result capture, etc.). All 437 real `.metta` files parse and translate successfully.

3.2 Sorry-Free Lean Proof

The translation is backed by a **sorry-free, axiom-free** Lean proof across two files totaling 4,979 lines and 102 definitions/theorems:

`HEPeTTaSound.lean` (1,063 lines, 44 theorems.) Proves that HE's equation matching corresponds to PeTTa's `ruleApp` evaluation. The key result:

> `simpleMatch_applyBindings_comm`: if HE's `simpleMatch` succeeds on two `PureTranslatable` atoms, then applying the translated bindings via `heBindingsToOSLF` to the translated pattern recovers the translated target.

This commutation property, combined with the existing `matchPattern_applyBindings_complete` from the OSLF framework, establishes that a PeTTa `matchPattern` witness exists whenever HE's `simpleMatch` succeeds.

`HEPeTTaTranslate.lean` (3,916 lines, 58 theorems.) The executable translator with:

- `translateHE_translatable`: translation preserves `Translatable` (output admits `atomToPattern`).
- `translateHE_preserves_pureTranslatable`: translation preserves the stronger `PureTranslatable` fragment.
- `translatePeTTa_translatable`: same for the reverse direction.
- 9 `#eval` tests matching Prolog output on concrete atoms.
- 6 `rfl` examples kernel-verified by Lean.
- Roundtrip idempotence examples (kernel-verified).

4 Cross-Runtime Benchmarks

All benchmarks use dialect-agnostic MeTTa (shared syntax) unless noted. Output is verified identical across runtimes.

4.1 Shallow Evaluation

Nil Geisweiller's typed backward chainer over 30 entities with 3 binary rules at depth 1. All three runtimes produce identical 30 proof-carrying witnesses, e.g. `(: (r-mortal r01 r01c) (mortal r01))`.

Runtime	Time	vs. CeTTa	Output
CeTTa	15 ms	1×	30 witnesses
PeTTa	151 ms	10× slower	30 witnesses
HE	1,010 ms	67× slower	30 witnesses

Startup overhead. HE's 138 ms startup (Rust binary + Python stdlib) dominates small tests. CeTTa starts in 6 ms (precompiled stdlib blob). On computation-heavy benchmarks the startup amortizes, revealing the true compute speedup of ∼17×.

4.2 Deep Proof Search: PeTTa Wins

Metamath proof search at depth 5 (2 axioms, proving $t = t$, 85,013 proof trees) reverses the performance ordering:

Runtime	Time	vs. PeTTa	Status
PeTTa	31.7 s	1×	85,013 proofs
CeTTa	94.2 s	3.0× slower	85,013 proofs
HE	>120 s	timeout	killed

PeTTa's Prolog SLD resolution with first-argument indexing is 3× faster on deep, branching proof search. HE cannot finish within 120 s.

4.3 Forward Chaining at Scale

Nil Geisweiller's propositional-calculus forward chainer at depth 4 (∼11.7M generated pairs) completes in 120 s on CeTTa. HE times out.

4.4 Translator Validation

A PeTTa-native benchmark using `prog1` (PeTTa-specific construct) was translated PeTTa→HE and run on CeTTa. The translator converts `prog1` to nested `let` with fresh variables:

```
; PeTTa original:
(= (run-query) (prog1 (bc &kb ...) (println! "done")))
; Translated to HE:
(= (run-query) (let $__tr_result_1 (bc &kb ...)
               (let $__tr_discard_2 (println! "done")
                 $__tr_result_1)))
```

The translated version produces identical output on CeTTa and HE, matching the PeTTa original.

4.5 PLN Evidence Aggregation over Genomic KB

To benchmark inference over real data, we run PLN evidence aggregation over the Rejuve genomic knowledge base (chr16 pilot: 41 GWAS SNPs, 183 SNP–gene pairs, 3 evidence channels). Each pair carries eQTL tissue counts (GTEx, 49 tissues), ABC enhancer–gene contact counts, and a regulatory presence flag. The PLN evidence quantale combines channels via revision (`evidence-hplus`), producing a calibrated SimpleTruthValue per pair. The evidence operations are Lean-verified: `power`, `toLogOdds_power_mul`, and `power_power_inv` are proved in `Mettapedia/Logic/BinaryEvidence.lean`.

The benchmark is dialect-agnostic (shared MeTTa, no translation needed). Both runtimes produce identical 183 scored pairs:

Runtime	Time	vs. CeTTa	Output
CeTTa	50 ms	1×	183 bio-scores
PeTTa	276 ms	5.5× slower	183 bio-scores

Scores carry genuine biological signal: `rs9923732→ENSG00000166455` scores strength 0.735 (131 ABC contacts across cell types), while most pairs score 0.04 (minimal evidence).

Scaling behavior. Synthetic stress tests reveal a crossover:

KB size	CeTTa	PeTTa	Winner
143	47 ms	193 ms	CeTTa (4.1×)
1,000	20 ms	271 ms	CeTTa (13×)
10,000	336 ms	1,616 ms	CeTTa (4.8×)
100,000	27.1 s	14.9 s	PeTTa (1.8×)
1,400,000	386 s†	132 s	PeTTa (2.9×)

†After bulk-load optimization: loading 5 s (faster than PeTTa), inference 381 s (O(N) match scan).
Pre-optimization: >1200 s timeout during loading.

CeTTa wins on small knowledge bases (<10K facts) due to fast startup and equation dispatch. PeTTa wins on large KBs (>50K facts) because Prolog's first-argument indexing gives O(1) fact lookup, while CeTTa's `match &self` performs O(N) linear scan. The pre-interning optimization (Section 5) would improve CeTTa's constant factor but not the asymptotic scaling; indexing the match backend for `match &self` queries is the structural fix.

4.6 Genomic Pattern Mining

To exercise real algorithmic depth, we run the Hyperon pattern miner [1] on real GTEx eQTL data, discovering surprising co-regulation patterns—pairs of SNPs or genes that share eQTL evidence more often than expected by independence. The data is extracted from the Rejuve genomic KB (338M Prolog lines, 57 GB) and discretized into categorical attributes (tissue breadth, effect size) for pattern mining.

Atoms	PeTTa	CeTTa	Patterns	Mode
200	—	268 ms	29	specialization only
1,000	16 s	1.1 s	61/20	full / with surprisingness
5,000	—	11.6 s	574	specialization only
10,000	271 s	∼23 s	883	with surprisingness
20,000	580 s	—	3,002	frequent only

CeTTa runs the miner pipeline ∼12× faster than PeTTa at comparable scale. At 10K atoms the miner discovers 883 surprising co-regulation patterns in 4.5 minutes on PeTTa vs. ∼23 s on CeTTa. These are genuine biological findings: shared cis-regulatory control, detectable only by cross-evidence pattern analysis.

The miner's Python helper module (`freq-pat`: De Bruijn index conversion, redundancy removal) loads on CeTTa via its Python FFI bridge, which provides a compatibility layer for the `hyperon` Python API. The miner's `build-specialization` function now runs on CeTTa, producing identical specialization patterns. A discovered interaction between `superpose` and mutable state—where HE interleaves side effects across non-deterministic branches, losing atoms—is correctly handled by CeTTa's sequential-per-iteration evaluation (Section 7). The *inference* phase—using discovered patterns as PLN rules for gene prioritization—runs on both runtimes (CeTTa: 76 ms, PeTTa: 193 ms on the focused 143-atom KB).

4.7 Backward Chaining over Mined Patterns

The mined co-regulation patterns serve as typed inference rules for Geisweiller's `nilbc` backward chainer, closing the *mine→chain* pipeline. The knowledge base combines:

- 100,000 real GTEx eQTL links (40,484 SNPs × 14,043 genes),
- 13 co-regulation patterns and 7 co-expression patterns (mined, surprisingness > 0.98),
- 3 typed PLN rules: *direct-target* (eQTL → target), *coreg-transfer* (co-regulation + target → target), and *coexpr-transfer* (target + co-expression → target).

Bridge base cases let `nilbc` consume untyped data atoms (`eqtl-link`, `co-regulate`) directly, avoiding a format conversion pass over the 100K-atom KB.

At depth 2, backward chaining discovers **5 novel gene targets** unreachable by direct eQTL lookup:

Gene	Symbol	Via SNP	Evidence path
ENSG00000150961	SEC24D	rs10000795	`(ct cr3 (dt eq37))`
ENSG00000172399	MYOZ2	rs10000795	`(ct cr3 (dt eq39))`
ENSG00000178636	MRPL42P1	rs10000726	`(ct cr1 (dt eq30))`
ENSG00000248394	FOSL1P1	rs10001048	`(ct cr25 (dt eq87))`
ENSG00000273077	(lncRNA)	rs10001048	`(ct cr25 (dt eq89))`

The proof terms are machine-checkable evidence chains: `(ct cr3 (dt eq37))` reads "co-regulation transfer via `cr3` (rs10000544↔rs10000795), applied to direct eQTL `eq37` (rs10000795→SEC24D)." For rs10000620, the chainer finds 4 independent evidence paths per gene (1 direct + 3 co-regulation partners), producing 32 proof terms suitable for PLN revision.

Query SNP	Genes	Novel	Proofs	Time (PeTTa / CeTTa)
rs10000544	12	3	96	
rs10000620	5	0	32	193 ms / 76 ms
rs10000960	8	2	14	

Biologically, the novel discoveries include MYOZ2 (hypertrophic cardiomyopathy gene, OMIM CMH16) and SEC24D (Cole-Carpenter bone syndrome), both on chr4q26—suggesting a trans-regulatory link from a metabolic locus (PDE5A/FABP2) to cardiac and skeletal gene targets via shared cis-regulatory architecture.

The full 100K-atom benchmark with all 7 query SNPs at depth 2–3 plus reverse queries completes in 63 s on PeTTa (564 hypotheses).

Complete benchmark ladder. The following table summarizes all benchmarks, including both genomic inference (where CeTTa wins) and pure search (where PeTTa wins):

Benchmark	Scale	CeTTa	PeTTa	Speedup
*Genomic inference (*CeTTa *wins)*				
Focused BC	143 atoms	66 ms	216 ms	3.3×
PLN evidence	143 atoms	61 ms	177 ms	2.9×
Pattern inference	143 atoms	17 ms	194 ms	11.4×
PLN 100K	128K atoms	13.9 s	20.8 s	1.5×
Deep 9-rule BC	261K atoms	3m 22s	5m 24s	1.6×
Depth-10 PLN	**1.4M atoms**	**15 s**	**143 s**	**9.5×**
*Pure search (*PeTTa *wins)*				
8-Queens (92 solutions)	8×8	2.75 s	0.21 s	13× slower
BTree 4,095	on-the-fly	0.87 s	0.15 s	5.8× slower
BTree 16,383	on-the-fly	24.7 s	0.14 s	176× slower
Tilepuzzle (181K)	181K states	3m 14s	6.4 s	30× slower

CeTTa wins decisively on inference-heavy workloads: the depth-10 PLN benchmark produces 19,845 hypotheses with calibrated truth values (including 1,632 depth-10 drug hypotheses at strength 0.084) in 15 s vs. PeTTa's 143 s—a **9.5**× speedup. PeTTa's Prolog WAM wins on pure search due to trail-based rollback, first-argument clause indexing, and compact term representation. The BTree 16,383 gap (176×) exposes CeTTa's remaining equation dispatch bottleneck: 80% of candidates still use linear scan.

Multi-relational deep chaining. Extending the rule set from 3 to 6 rules—adding Reactome pathway membership, same-pathway gene, and pathway-mediated target—enables cross-ontology chains at depth 4: SNP $\xrightarrow{\text{coreg}}$ SNP′ $\xrightarrow{\text{eQTL}}$ Gene $\xrightarrow{\text{pathway}}$ Pathway $\xrightarrow{\text{co-member}}$ Gene′. The 6-rule chainer over 100K eQTL + 943 pathway edges produces **10,036 hypotheses** (256 unique genes, 62 pathways) in 24 s on PeTTa. Each hop introduces a genuinely different relation type, preventing circular chains and enabling the kind of multi-source inference that single-relation depth cannot achieve.

Adding Hetionet disease associations, compound–target bindings, and scaled Reactome pathways (132,556 edges, 35,738 genes) extends the chain to the full variant-to-drug pipeline. A 9-rule chainer produces **1,411,430 scored hypotheses**—identical on both runtimes—in **3m 22s on CeTTa** vs. 5m 24s on PeTTa (1.6× speedup), including 295K depth-3 pathway targets, 797K depth-4 pathway targets, 488 drug hypotheses, and 17,429 unique reachable genes from 47 direct targets. The textbook chain rs10000544 → PDE5A → hypertension ← Sildenafil composes three independent data sources (GTEx, Hetionet, DrugBank) by typed backward chaining with proof terms at every step.

Dependent binder encoding. PLN rule declarations use `(rule-sig name type)` instead of `(: name type)` to avoid a current HE spec limitation: the `(: $e T)` syntax in arrow-domain positions is treated as a literal expected type, not a dependent binder, causing `BadArgType` on both HE and CeTTa (see Section 8). The `rule-sig` encoding is semantically equivalent on all runtimes.

5 Performance Analysis

5.1 SymbolId Conversion: 38% Instruction Reduction

The original CeTTa profiling revealed 46% of runtime in string operations (`intern`, `strcmp`, `atom_is_symbol`). We completed a comprehensive SymbolId conversion: all semantic dispatch—evaluator special forms, grounded operators, library hooks, registry lookups—now uses integer SymbolId comparison instead of string comparison.

Callgrind on the 8-puzzle BFS (5,000 steps) shows the impact:

	Before SymbolId	After SymbolId	Change
Total instructions	4,551M	2,810M	−38%
#1 hotspot	`intern` (18%)	`bindings_apply` (5.5%)	eliminated
#2	`atom_is_symbol` (11%)	`bindings_free` (5.4%)	eliminated
#3	`strcmp` (11%)	`resolve_refs` (4.4%)	eliminated

The remaining profile is uniform: no single function dominates. The top costs are now binding management (`apply`, `clone`, `free`) and heap allocation (`malloc`/`free`), pointing to trail/rollback as the next optimization target.

5.2 Tilepuzzle: 30× Gap with PeTTa

The 8-puzzle BFS (181,440 reachable states) is a WAM-class benchmark that exposes CeTTa's remaining architectural gap:

Runtime	Time	Memory	Status
PeTTa (Prolog/WAM)	6.4 s	114 MB	baseline
CeTTa (C/arena)	3m 14s	4.68 GB	30× / 41×

Massif heap profiling identifies the root causes: 44% of peak memory is arena allocation via `bindings_apply` (variable substitution creating fresh atoms with no hash-consing on the evaluation arena); 12% is `atom_expr` (expression construction); 7% is `rename_vars` (deep-copying equations for standardization-apart). Runtime counters show 7M `bindings_free`, 3.5M `bindings_clone`, and 1.4M `rename_vars` calls for 5,000 BFS steps.

5.3 Structured Space Kinds

CeTTa now supports three ordered space kinds beyond plain atomspaces:

Kind	Discipline	Use case
`queue`	FIFO push/pop	BFS frontiers, work queues
`stack`	LIFO push/pop	DFS, continuation stacks
`hash`	O(1) membership	Visited sets, deduplication

These are created via `(new-space queue)`, `(new-space hash)`, etc., and support universal pattern matching through the standard `match` API. The tilepuzzle uses queue-space for its BFS frontier and hash-space for visited-set deduplication, reducing peak memory from 6.3 GB (plain atomspace) to 4.68 GB.

5.4 Architecture: Three Machines

Profiling and design analysis (informed by GPT-5.4 Pro architectural review) identifies three semantically distinct machines that should not share mutable state:

1. **Local search machine.** Branch-local bindings, trail/rollback, choice frames, scratch arenas. WAM-inspired: cheap undo via watermarks, not clone/free churn. This is the next implementation priority.
2. **Indexed shared-space machine.** Persistent PathMap/MORK-backed spaces, snapshot/epoch discipline, conjunction query planning, leapfrog triejoin. Deterministic and committed—no backtracking through the index.
3. **Type/elaboration machine** (future). Scoped binders, metavariables, constraint solving, delayed assignments. Separate from runtime bindings.

Language profiles (`he_compat`, `he_extended`, `he_prime`, future `petta`) are compositional assemblies over these machines, not ad-hoc forks in the evaluator.

5.5 RhoCalc as a Reaction Machine

RhoCalc adds a fourth pressure point to the architecture: not a new storage backend, and not merely another syntax over atom-space evaluation, but an active process language whose basic unit of work is a *reaction*. The current CeTTa branch exposes it as `-lang rhocalc`, with both a Rholang-like `.rho` surface and a MeTTa-style `.mrho` surface. The `core` profile is treated as an explicit restriction, not as the identity of the language.

The executable kernel currently supports the small asynchronous spine:

$$0, \quad P \mid Q, \quad x!(v), \quad \mathsf{for}(p \leftarrow x)\{P\}, \quad \mathsf{new}\ x\ \mathsf{in}\ P, \quad @P, \quad *x$$

together with joins and persistent receives/contracts. Operationally, this is implemented by a `RhoMachine` object rather than by scattering rho-specific state through the ordinary evaluator. A machine slice carries its run budget, fresh-name counter, last run result, and reusable reaction-index capacity.

The important implementation distinction is between:

- **semantic freshness**: internally allocated rho fresh names are opaque grounded identities, printed as `rho:fresh:N` only for deterministic tests and human inspection; and
- **surface spellings**: a user-written symbol such as `rho:fresh:1` is not a capability and does not communicate with the internally allocated name that happens to print the same way.

Quoted process names are similarly treated as names with an opaque boundary. Substitution and pattern matching may compare quoted names as whole names, but they must not casually bind through the quoted process body. This is why the fast send index only indexes directly comparable names such as free symbols and opaque fresh identities; quoted process names remain on the fallback path where alpha-normalized process equality is used.

Indexed reactions. The first reaction engine is still single-threaded, but no longer treats every step as an undifferentiated scan. Each slice builds a live index of sends and reaction sites over the current normalized process, reusing the underlying buffers across steps. A receive or join first consults the indexed sends for direct channels and falls back to a full scan only for quoted or otherwise non-direct names. Persistent receives and joins reuse the same machinery while leaving their receive process in the residual process.

This design deliberately separates *persistent capacity* from *stale pointers*: the index is rebuilt against the current process after a reaction, but its allocated storage is retained. A later refinement can add stable component identities and delta updates, but the present design already removes repeated allocation and gives the runtime an explicit place to hang slice/resume and future parallel reaction scheduling.

Compatibility benchmark ladder. CeTTa compares its `.rho` and lowered `.mrho` execution against the local `f1r3node rholang-cli`. The benchmark script runs three stages: direct `.rho`, translated `.mrho`, and external `rholang-cli`. Ratios are reported only when both engines actually complete; if the external engine panics or times out, the comparison is marked `NA` rather than smuggling a failed run into a speed ratio.

Representative single-repeat measurements from the current branch:

Case	Size	CeTTa `.rho`	`f1r3node`	Note
pipeline	1024	5.14 s	16.94 s	linear receive/send chain
comm-pairs	256	0.82 s	2.05 s	independent communications
persistent-server	256	0.21 s	0.54 s	one contract serving many sends
join-pairs	256	1.73 s	4.91 s	two-channel joins
join-pairs	64	0.10 s	fail	external `TryFromIntError`

The persistent-server case is especially informative: it exercises the contract/persistent-receive path without requiring blockchain machinery, networking, or generalized Rholang effects. It is therefore a good early benchmark for whether the reaction engine handles reusable receive capacity without collapsing into materialized intermediate lists.

Relation to the rest of CeTTa. RhoCalc should teach the shared runtime about channels, sources, sinks, cancellation, and reaction scheduling, but it should not force every MeTTa operation into process-calculus overhead. The intended direction is to extract the reusable substrate—budgets, machine slices, indexed reaction state, cancellation tokens, and eventually worker pools—while keeping HE, PeTTa, and RhoCalc free to choose different surface semantics.

5.6 Faithful Backend Obligations

The indexed shared-space machine needs a sharper correctness contract than "the backend is fast and seems to return the same answers." Recent Lean work factors the backend story into three explicit runtime seams:

1. **Candidate selection.** The index narrows candidate atoms, while CeTTa's native matcher performs exact variable-sensitive matching. This is the two-phase architecture now validated by concrete selector theorems over PathMap tries and by a concrete matcher instantiation on the HE side.

2. **Canonical storage.** Storage may quotient results to canonical support, but it must preserve co-reference. Repeated source variables must remain repeated after storage and materialization; distinct variables must remain distinct.

3. **Revision-based caching.** Cached query answers are valid only across mutations that preserve the relevant support boundary. Duplicate adds or high-count decrements may preserve cache validity; first adds and last removes must invalidate. Variant-normalized cache keys must also preserve right-hand-side answers and result shape under materialization.

This yields a practical division of labor for the PathMap/MORK backend:

1. PathMap stores canonical support and performs prefix-based candidate narrowing;

2. the native matcher remains the source of truth for exact bindings and binder hygiene;

3. count metadata, not trie-key distortion, carries bag semantics.

Verified runtime contracts. Recent Lean work extends the backend correctness story with three new contract files:

- `CeTTaRuntimeContracts.lean` (20 theorems): canonicalization injectivity and BEq preservation (`term_canon.c`), search-context save/rollback correctness (`search_machine.c`), effect classification safety (`runtime_effect_classes.json`), and profile hierarchy (`session.c`). Each theorem maps to a concrete refactoring risk it reduces.

- `IncrementalTableSemantics.lean` (15 theorems): specifies partial-table soundness, answer monotonicity, and completion for future tabling in `table_store.c`. Any correct implementation must satisfy `TableEntry.Sound` (partial reads are genuine) and `TableEntry.Exact` (completed tables equal `queryEquations`).

- `ProducerChoicePoint.lean` (10 theorems): formalizes the search-machine save/rollback/nested-choice-point protocol with combined bindings+scratch state.

Crucially, CeTTa's imported backend is binding-correct not through trusted direct bridge bindings, but because candidate narrowing is followed by seeded native rematch (`space_subst_match_with_seed`). The theorem `imported_seedRematch_binding_parity` in `SeedRematchContract.lean` formalizes this: when imported candidates are sound, the two-phase query produces the same result set as direct matching.

Positive example. Adding an existing atom to a counted space may increment multiplicity without changing trie support or invalidating unrelated cached queries.

Negative example. Letting the backend bridge perform full unification over serialized query terms invites exactly the binder-hygiene failures that arise when canonicalization preserves shape but breaks co-reference.

These obligations are now mapped one-for-one to runtime seams: `space_match_backend.c` for candidate selection, `term_universe.c` for canonical storage, and `table_store.c` for cache invalidation. The companion note *Faithful Backends for HE MeTTa* develops the full proof story, including the variant-query and binding-cache lemmas; the present paper uses those results to justify the architecture of CeTTa's indexed shared-space machine.

5.7 MM2 Integration

MM2 is a deterministic, priority-scheduled, forward-chaining term rewriting language implemented by MORK. Its programs are not a separate execution layer—they are atoms in MORK-backed spaces, stepped by MeTTa via `step!`. The hybrid interaction requires no special protocol: MeTTa creates spaces, adds data, steps MM2 rules, and reads results through ordinary `match`.

```
!(bind! &ws (new-space mork))
!(add-atom &ws (exec (0 step)
  (, (edge $x $y) (edge $y $z))
  (0 (+ (path $x $z)))))
!(add-atom &ws (edge a b))
!(add-atom &ws (edge b c))
!(step! &ws)
!(match &ws (path $x $y) ($x $y)) ; -> (a c)
```

Three import paths exist: `.mm2` file import (`import!` or `include` into a target space, which auto-enables the MORK backend), inline `add-atom` of exec rules from MeTTa, and compiled `.act` artifact loading via `mork:open-act`.

Because MM2 rules are ordinary atoms, MeTTa can pattern-match over them as data—inspecting priorities, rewriting source/sink templates, or extracting rules for analysis:

```
!(match &ws (exec $p $src $snk) (rule $p))
```

CeTTa's `mm2_lower.c` handles bidirectional surface↔IR lowering (`exec`↔`mm2_exec`, `,`↔`mm2_pattern_and`, etc.), preserving round-trip fidelity between MM2 surface syntax and CeTTa's internal representation.

6 The MeTTa Dialect Landscape

6.1 Shared Fragment

85% of MeTTa programs use only shared constructs (`let`, `let*`, `match`, `if`, `case`, `bind!`, `add-atom`, `new-space`). These run identically on all three runtimes without translation. The Metamath proof search benchmark is dialect-agnostic.

6.2 Translated Constructs

Direction	Construct	Translation
HE→PeTTa	`chain`	`let`
	`collapse-bind`	`collapse`
	`superpose-bind`	`superpose`
	`nop X`	`let $fresh X ()`
	`function (return X)`	`X` (unwrap)
PeTTa→HE	`progn A B`	`let $fresh A B`
	`prog1 A B`	`let $r A (let $d B $r)`
	`@<`	`<s`

6.3 Semantic Divergence: Constraint Store vs. Independent Scope

The translation preserves semantics on the *pure fragment* (ground terms, or variables scoped by `let*`). A genuine divergence exists for free variables in non-deterministic branches:

```
(= (deduce (And $a $b)) (And (deduce $a) (deduce $b)))
```

When `deduce` returns via `match &self` with a free variable `$x`, HE/CeTTa bind `$x` consistently across both branches of `And`, while PeTTa gives each branch its own copy.

This divergence has a precise theoretical characterization. Saraswat et al. [2] showed that concurrent constraint languages based on Tell-and-Ask over a shared store satisfy a *monotonicity property*: constraints accumulate monotonically, and any reduction possible in a later configuration was already possible in an earlier one (their Proposition 2.1). HE/CeTTa's `match &self` operates on a shared constraint store—bindings produced by one branch propagate to all others via the shared space. PeTTa's Prolog backend evaluates branches with independent variable scopes, closer to what Saraswat et al. identify as the "read-only variable" model of Flat Concurrent Prolog, which they explicitly note does *not* possess the monotonicity property.

The depth-1 standard form. Saraswat et al. define a clause to be in *standard form* when each atom in the body is depth-1 (no shared variables between conjunction branches). Nil Geisweiller's typed backward chainer uses exactly this form: `let*` threads variable bindings explicitly between premises, eliminating implicit sharing. This is why the typed chainer works identically on all three runtimes—it stays within the standard-form fragment where the constraint-store and independent-scope semantics coincide.

Lean formalization. The `Translatable` and `PureTranslatable` predicates in `HEPeTTaSound.lean` characterize precisely this fragment: atoms where `atomToPattern` succeeds and produces `isMatchCorrect` patterns (no lambda, subst, or collection nodes). The sorry-free proof establishes that equation-matching semantics are preserved under translation on this fragment. Extending the proof to the shared-variable conjunction fragment would require formalizing the monotonic constraint-store semantics of HE's `match &self`—a natural target for Phase 3 of the Lean proof.

7 Non-determinism and Mutable State

A fundamental specification gap in MeTTa concerns the interaction of non-deterministic evaluation with mutable space operations. When `let $x (match &db ...) (add-atom &result (stored $x))` executes with multiple match results, what happens to the shared `&result` space?

7.1 The Problem

Consider:

```
!(bind! &db (new-space))
!(add-atom &db (item A))
!(add-atom &db (item B))
!(bind! &result (new-space))
!(let $x (match &db (item $y) $y)
   (superpose ((remove-atom &result (stored $x))
               (add-atom &result (stored $x)))))
!(get-atoms &result)
```

HE returns `[]`—all atoms lost. CeTTa returns `[(stored A) (stored B)]`—correct. The root cause: HE interleaves side effects from different non-deterministic branches, so `remove-atom` from one iteration cancels `add-atom` from another. CeTTa evaluates each iteration's body to completion before starting the next. This is the exact pattern used by the Hyperon pattern miner's `build-specialization`, explaining why the miner produces empty results on HE.

7.2 Four Possible Semantics

We analyzed four options across eight representative examples (accumulation, logging, constraint checking, search with backtracking, speculative execution, order-dependent processing, accidental non-determinism, and the real miner code):

Option	Wins	Loses	Character
Sequential (CeTTa)	6/8	1/8	accumulation-friendly
Interleaved (HE)	0/8	4/8	always worse than sequential
Forked (copy-on-write)	1/8	4/8	search-friendly
Illegal (UB)	1/8	5/8	safest, least ergonomic

Sequential evaluation loses only on *search with backtracking*, where each branch needs isolated state. Interleaved evaluation is strictly dominated—it provides no advantage over sequential while introducing race conditions on shared mutable state.

7.3 `with-space-snapshot`: The Escape Hatch

For the one case where sequential semantics is wrong (search with mutable state), CeTTa provides `with-space-snapshot`:

```
(= (fs-add-to-ss $rb $ss)
   (with-space-snapshot $ss0 $ss
     (match $ss0 (: $x1 $a1)
       (match $ss0 (: $x2 $a2)
         (match $rb (: $f (-> $a1 $a2 $b))
           (add-atom $ss (: ($f $x1 $x2) $b)))))))
```

This pattern, from Geisweiller's propositional forward chainer, reads from a frozen snapshot `$ss0` while writing to the live space `$ss`—the standard *semi-naive* discipline for stratified forward inference. This is a CeTTa extension; HE does not currently implement it.

7.4 Proposed Spec Clarification

The MeTTa spec [1] uses sequential Python pseudocode (line 368: a list comprehension over results) that implies sequential side-effect ordering, but does not explicitly require it. We propose:

> *When a non-deterministic expression produces multiple results and those results are bound via `let`/`chain`/equation dispatch, the body for each result is evaluated to completion—including all side effects—before the body for the next result begins. To evaluate branches with isolated state, use `collapse` first, then iterate explicitly.*

A future `snapshot-space` primitive would provide clean copy-on-write semantics for search/backtracking without baking merge semantics into the language core.

8 HE′: Dependent Telescope Extension

Both HE and CeTTa treat `(: $e T)` in arrow-domain positions as a literal expected type, not a dependent binder. This means Curry-Howard backward chainers cannot store rule types as `(: dt (-> (: $e (eqtl $snp $gene)) (target $snp $gene)))` in `&self` without triggering `BadArgType`. This is a current HE spec limitation, not a CeTTa bug.

We formalize an explicit extension, HE′ (he_prime), as a *telescope interpretation* of arrow domains. The extension is exactly one function: `splitDomain`, which recognizes `(: $x T)` as a dependent binder instead of a literal. Everything else (matching, bindings, space operations, equation dispatch) is inherited from HE unchanged.

The telescope walk checks arguments left-to-right: each successful type match extends the binding environment, and later domains and the codomain are instantiated under accumulated bindings. This is formalized in Lean (230 lines, 0 sorries) at `HEPrime/Telescope.lean`, with kernel-checked positive and negative examples:

- **Positive:** applying `dt` to `eqtl_rs1_gene1` infers return type `(target rs1 gene1)` via dependent substitution. Binary rule `ct` with cross-domain dependency (`$snp2` bound by first argument, constraining second domain) also kernel-checked.
- **Negative:** mismatched SNP (`rs2` ≠ `rs1`) correctly produces `none`. Standard HE telescope parsing treats the same domain as a plain atom, proving the semantics are distinct.

CeTTa implements HE′ as an opt-in profile (`he_prime`), gated by a single flag (`enable_dependent_telescope`). The implementation mirrors the Lean formalization: `split_dependent_domain`, `function_domain_type`, `bind_domain_binder`, and `infer_dependent_application_types` in `eval.c`.

9 Related Work

Concurrent constraint programming. Saraswat et al. [2] established the theoretical framework for detecting stable properties of networks in concurrent logic programming languages. Their cc(↓,|) language, with Tell/Ask on shared variables over a monotonic constraint store, provides the semantic model that most closely matches HE/CeTTa's `match &self` evaluation. The distinction between monotonic constraint stores and independent-scope evaluation (Flat Concurrent Prolog) precisely characterizes the semantic boundary between HE and PeTTa.

MeTTa ecosystem. The MeTTaIL boot compilation pipeline [3] provides a contract-first, Lean-backed compilation path for MeTTa stdlib modules, complementing CeTTa's direct evaluation approach. The Mettapedia formalization project provides the Lean specifications (`EvalSpec`, `MeTTaEval`, `MatchSpec`) that ground both the translation proof and the CeTTa evaluator design. The PLN (Probabilistic Logic Networks) framework uses MeTTa's backward chaining extensively; the typed chainer pattern from Nil Geisweiller's `nilbc.metta` is the standard PLN idiom that works across all three runtimes.

10 Conclusion

CeTTa demonstrates that a direct C implementation of the MeTTa evaluator can achieve **9.5× speedup** over PeTTa on genuine depth-10 PLN backward chaining with truth-value propagation (19,845 hypotheses, 1,632 depth-10 drug candidates, 15 s vs. 143 s), while maintaining oracle-verified

correctness across 297 tests. The sorry-free Lean proof of HE↔PeTTa semantic correspondence (5,400+ lines, 0 sorries), including import commutativity and operational bridges for state and space allocation, establishes that MeTTa's three runtimes are semantically compatible on the shared fragment. The compiled `.act` space format via MORK enables zero-copy indexed loading of large knowledge bases.

The 30× gap with PeTTa on the 8-puzzle BFS benchmark is precisely diagnosed by callgrind/-massif profiling: binding clone/free churn (7M free calls for 5K steps) and equation deep-copying (1.4M `rename_vars` calls). These are addressable by WAM-inspired trail/rollback without converting CeTTa into a Prolog engine. The guiding principle: *backtracking should be rare, local, and hidden—not global, ambient, or user-visible.*

Future work.

- **Local search machine:** trail/watermark rollback, `BindingsBuilder`, arena mark/reset for scratch atoms. Target: 3–5× on search-heavy workloads (tilepuzzle gap from 30× to ∼6–10×).
- **Indexed shared-space engine:** leapfrog triejoin (from Vandervorst's `trie_synth.py`), conjunction query planning, SpaceEngine boundary (formalized in `SpaceEngineBoundary.lean`: native ⊂ pathmap ⊂ mork, with ACT as artifact-only surface; `ACTArtifactBoundary.lean` classifies `open-act/load-act!/dump!` as storage seams). Target: 10–100× on large-KB multi-pattern conjunctions.
- **MM2 as MeTTa data (working):** MM2 exec rules are atoms in MORK-backed spaces. MeTTa creates, inspects, rewrites, and steps them through the ordinary space API. `step!` is the execution interface; `.mm2` files import via `include/import!`. MeTTa can pattern-match over MM2 code to analyze or transform programs—the meta-language relationship is native.
- **morkl: expert PathMap algebra (future):** Direct access to PathMap's zipper navigation (5,800 lines of Rust), product-zipper Cartesian joins (2,000 lines), and lattice operations (`pjoin/pmeet/psubtract/prestrict`, 1,600 lines). MORK's kernel already implements 15+ sink types (count, sum, head, hash, and, Z3, WASM) and source types (BTM, ACT, equality/inequality constraints). The Lean formalization (46 files, 19K+ lines) proves the three-phase execution protocol, fold-level aggregation, and PathMap lattice correspondence. The gap is FFI exposure and MeTTa surface: the Rust substrate exists, `lib/morkl.metta` does not yet.
- **HO search engine:** Lash-inspired SAT-driven proof search for PLN and type-directed synthesis. PicoSAT as embedded SAT solver. Separate from the evaluator.
- **Translator CLI:** `translate.sh` with `he2petta/petta2he`, recursive and bundle modes. Lean proof covers pure fragment + state + spaces (0 sorries).
- Full pattern miner on CeTTa end-to-end (∼12× speedup over PeTTa).

A Experimental — MAM: The MeTTa Abstract Machine

This appendix is explicitly experimental. It captures design exploration from ongoing discussions (April 2026) between the authors, external reviewers (GPT-5.4 Pro), and Codex, concerning the post-phase-1f abstract-machine layer for CeTTa*. Nothing here is yet implemented or committed*

architecture. It is recorded to preserve design rationale; the authoritative implementation plan remains `implementation_plan.tex`.

A.1 Motivation

The phase-1f-era extraction seam (`src/search_machine.{h,c}`) pulls policy parsing, stream combinators, and a `TableStore` plugin out of `eval.c`, and wraps them in a thin callback struct `CettaSearchMachine = {stream_source, eval_bound_single, eval_ctx, table_plugin}`, stack-allocated at each call site. This is a useful refactor, but it is a *plug interface* — not an abstract machine in the WAM / ZAM / SLG-WAM sense. Real abstract machines (XSB's SLG-WAM, SWI's tabling runtime, Soufflé's compiled Datalog, miniKanren's CPS-transformed search) have persistent machine state, explicit visitor protocols, capability advertisement, and episode lifetimes.

The proposal here is a *principled* next layer that retains the current code as `SearchAdapter` (transitional helper) while naming the real abstraction from first principles.

A.2 Current categorical reading

The MAM's present semantic reading, under active revision, models it as a **quantale-enriched traced profunctor**, fibrated for reflection. This reading is provisional — it captures the current best understanding of the structure, not a definitional claim that binds future architecture. Unpacked:

- **Profunctor** $P : \mathcal{Q}^{\mathrm{op}} \times \mathcal{A} \to \mathcal{V}$: *search*. Queries in $\mathcal{Q}$ relate to answer streams in $\mathcal{A}$, with intensity weighted by elements of $\mathcal{V}$.
- **Quantale enrichment** $\mathcal{V} = (L, \otimes, \leq, \bigvee)$: *multiplicity*. Answer weights live in a quantale: $\mathcal{V} = \mathbf{2}$ for classical boolean search; $\mathcal{V} = \mathbb{N}$ for counted answers; $\mathcal{V} = [0,1]^2$ (strength $\times$ confidence) for PLN. PathMap's trie-algebra is natively a quantale.
- **Trace** $\mathrm{Tr}_X^{A,B} : \mathrm{Hom}(A \otimes X, B \otimes X) \to \mathrm{Hom}(A, B)$: *fixpoint*. Feedback / saturation / tabling / cyclic queries are all instances of categorical trace (Joyal–Street–Verity, Hasegawa).
- **Kleisli structure over an effect monad** (or algebraic effect handlers *à la* Plotkin–Pretnar): *control*. Demand, cut, suspension, resumption, and schedule are composable effect operations.
- **Fibration** $p : \mathcal{T} \to \mathcal{B}$ (optional, for reflective MeTTa): rules-over-machine-state.

A trace operator is provably equivalent to taking the least fixpoint of a parameterised endofunction (Hasegawa 1997); this is why tabling, PLN-convergence, and saturation all fit one uniform operator.

A.3 The proposed machine family: MAM

We name the machine family **MAM: MeTTa Abstract Machine**. The name is chosen by identity rather than by current structure or scope: MAM is the abstract machine for the MeTTa language family, and that identification remains accurate as the internal architecture evolves. The naming tradition (WAM, ZAM, STG, CEK, CAM) uses either author identity or component decomposition; MAM uses the language identity it serves, which is stable across architectural iteration. Specifically, MAM does *not* hardcode a categorical claim (as "QAM = Quantale Abstract Machine" would

have done), and does *not* hardcode a functional scope (as "SCAM = Search & Control Abstract Machine" would have done). The categorical reading and the current feature set are both free to evolve without renaming.

Concept	Name
Machine family	`MAM` (Quantale Abstract Machine)
Formal description	Quantale-enriched traced profunctor
Engine 1 (goal-directed, Kleisli/effect)	`Chainer`
first realization (native)	`native_chainer`
Engine 2 (fixpoint, trace)	`Saturator`
first realization (pathmap)	`pathmap_saturator`
Per-session state	`Episode`
Dispatch vtable	`MAMOps`
Answer	`Answer` / `WeightedAnswer`
Schedule axis	`dfs` / `bfs` / `iddfs` / `interleave` / `native`
Demand axis	`unbounded` / `once` / `limit-k` / `fold`
Visitor control	`CONTINUE` / `STOP` / `ERROR` / `SUSPEND`

A.4 The two-engine pressure strategy

Rather than over-specifying a first concrete machine, MAM is pressured from day 1 by *two* independent realizations that stress complementary axes of the seam:

`Chainer` (goal-directed). WAM-lineage control: explicit environments/frames, choicepoints, trail/undo, deterministic tail-path handling, bounded-demand response, direct conjunction when advertised. Design-guide workload: **metamath backward chaining** (where PeTTa currently completes and CeTTa OOMs). First realization: `native_chainer`.

`Saturator` (fixpoint). SLG / Datalog / saturation lineage: snapshots, delta-sets, conjunction planning, exact membership, counted answers, eventual path toward PLN and ATP-style saturation. Design-guide workload: **PLN multi-pattern deduction**. First realization: `pathmap_saturator` (because PathMap's quantale structure makes weighted answer aggregation natural).

This pair creates the right kind of pressure on the seam: Chainer forces frames, choicepoints, `EnvRef`, demand handling, and explicit scheduling; Saturator forces snapshots, delta sets, conjunction planning, exact membership, counted answers, and the eventual path to PLN and ATP.

Compiled / bytecode (ZAM-lineage) and ATP saturation engines are explicit *later* lanes — they risk freezing the protocol in the wrong shape if pursued before Chainer and Saturator coexist cleanly under one MAM.

A.5 Axes that must stay separate

A core architectural commitment is that the following axes are **distinct data** in `SearchRequest`, not tangled behind surface-level operators:

Schedule. `dfs` / `bfs` / `iddfs` /`interleave` are *fairness policies*, not engines. Korf's classic result motivates `iddfs` as a completeness-friendly default over raw `bfs` (BFS completeness with DFS memory); miniKanren's interleaving exists precisely to avoid starvation on infinite branches.

Demand. `once` / `limit-k` / `unbounded` /`fold` describes how many answers are requested, independently of schedule. `once` and `select 1` must build `demand = FIRST`; they must *not* silently hardcode DFS.

Table mode. `none` / `variant` / `subsumptive` /`incremental` mirrors the XSB / SWI ladder; each is an explicit engine-level choice.

Backend. `native` / PathMap / MORK chosen by the actual `Space`, not by a global preference.

Lane. `recursive-dependent-proof` / `atp-saturation` /`solver-oracle` / ... chosen at surface.

The request builder (one per `-lang`) normalizes surface syntax into `SearchRequest + SearchLangContract`; the MAM consumes the normalized object, never raw surface atoms.

A.6 The seven-type vocabulary

Seven types are proposed to land together before the first real backend optimization (counted-answer variant table, PathMap-native conjunction, native exact-match index). Landing them simultaneously prevents the "first optimization forces a rewrite" pathology.

Type	Categorical role
`SearchRequest`	Object of $\mathcal{Q}$
`SearchAnswer`	Object of $\mathcal{A}$ (quantale-weighted)
`SearchLangContract`	Subcategory inclusion $\mathcal{C} \hookrightarrow$ MAM
`SearchCaps`	Capabilities = presence of morphisms/natural transformations
`VisitCtl`	Algebra of the effect operation visit
`MAMOps`	Kleisli dispatch vtable
`Episode`	One trace-iteration instance

Implementation-actual `SUSPEND/RESUME` behavior, cancellation tokens, snapshot-token machinery, and per-`-lang` registration remain explicitly deferred.

A.7 Reference base

The proposal draws on the following literature, all available in the local library:

- *Aït-Kaci 1991, WAM Tutorial Reconstruction* – foundational WAM semantics; drives `Chainer` design.
- *Swift & Warren, XSB System Manual; Sagonas et al. 1994* – SLG-WAM tabling; drives `Saturator` / table-mode ladder.
- *Scholz, Jordan, Subotić, Westmann 2016, Soufflé: On Fast Large-Scale Program Analysis in Datalog (CC)* – staged compilation of fixpoint computation; compiled `Saturator` path.
- *Keoliya & Byrd 2020, microKanren* – interleaving search as a distinct schedule axis.
- *Ager, Biernacki, Danvy, Midtgaard 2003, A Functional Correspondence Between Evaluators and Abstract Machines (BRICS)* – derivation methodology for machines from evaluator semantics.

- *Leroy 1990, The ZINC Experiment* – machine-description vs. run-instance separation (maps to `MAMOps` / `MAM` / `Episode`).
- *Russell & Norvig, AIMA Ch. 3* – IDDFS completeness; uniform treatment of uninformed search.

A.8 Status

This appendix records design rationale; it does not commit implementation. Reasoned council review converged on ∼0.88 confidence on MAM naming and two-engine strategy. The concrete implementation plan (phase 2 seam; phase 3 tabling plugin; phase 4 backend parity) is the authoritative schedule and is unchanged by this appendix.

This text should be revisited at each phase boundary and revised as implementation reveals which proposals held and which did not.

References

[1] B. Goertzel et al., "MeTTa: A Reflective Language for AGI," OpenCog Foundation, 2024.

[2] V. Saraswat, D. Weinbaum, K. Kahn, and E. Shapiro, "Detecting Stable Properties of Networks in Concurrent Logic Programming Languages," in *Proc. 7th ACM Symposium on Principles of Distributed Computing (PODC)*, pp. 210–222, 1988.

[3] Z. Goertzel and Oruži, "MeTTaIL: Contract-First Boot Compilation for Direct C MeTTa Runtimes," Working paper, 2026.